Agenda

- Syllabus Update
- Have you started HW1?
- Coroutines
- Channels
- Coroutine implementation

Clarification

The exact behavior of yield (does it **have** to switch?) is defined by the specific coroutine library

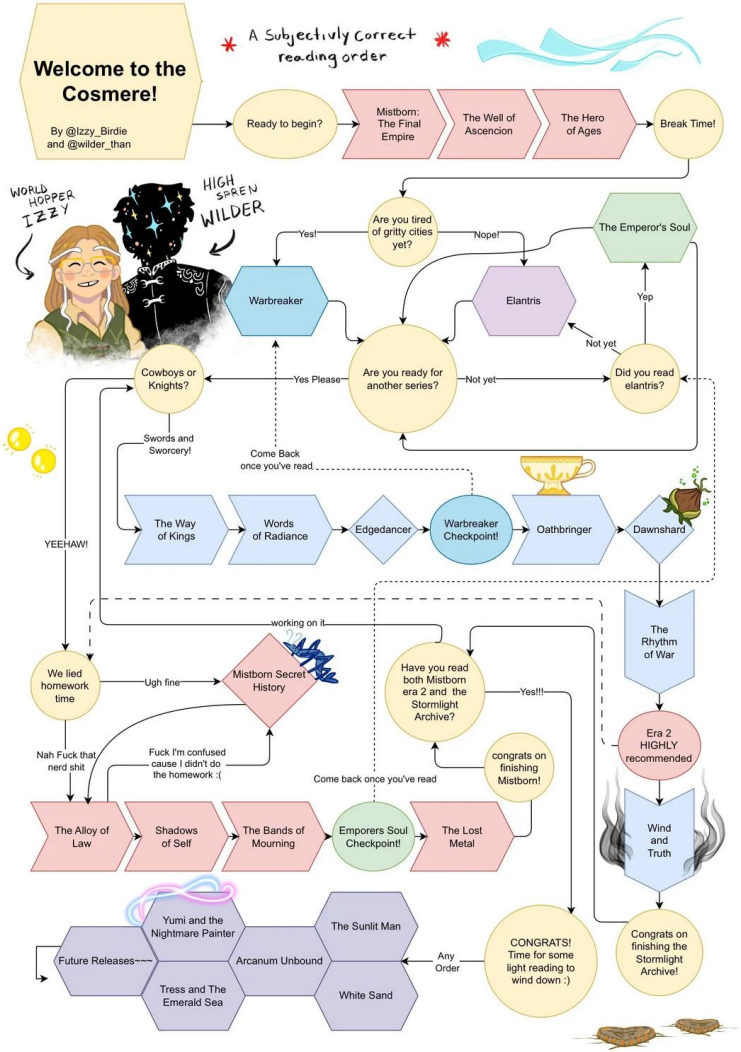
HW1: Yes

Code examples today: No (more general)

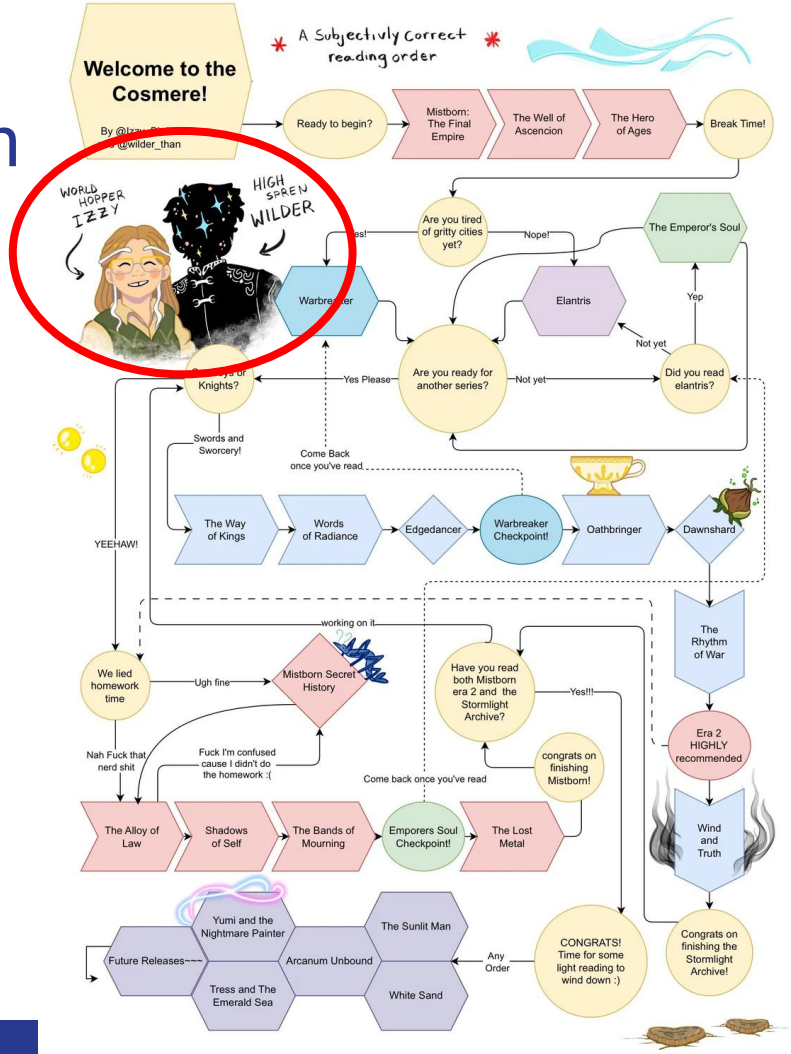
Channels

- A crucial part of coroutines
- How any useful work gets done!

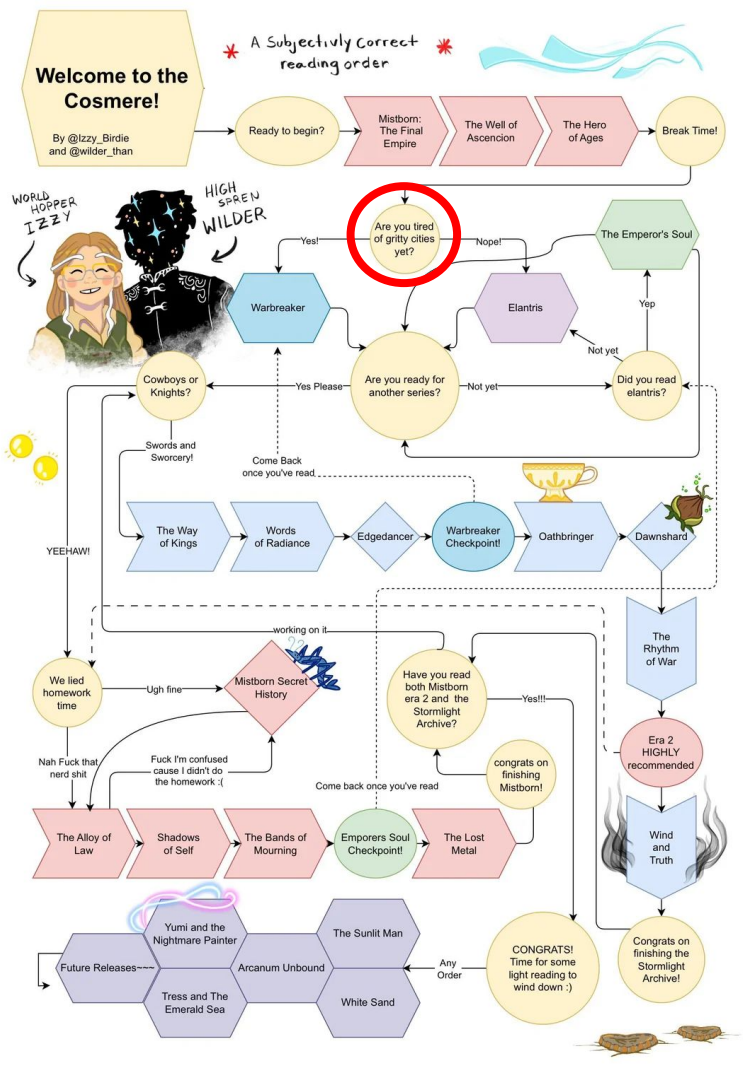
Brandon Sanderson



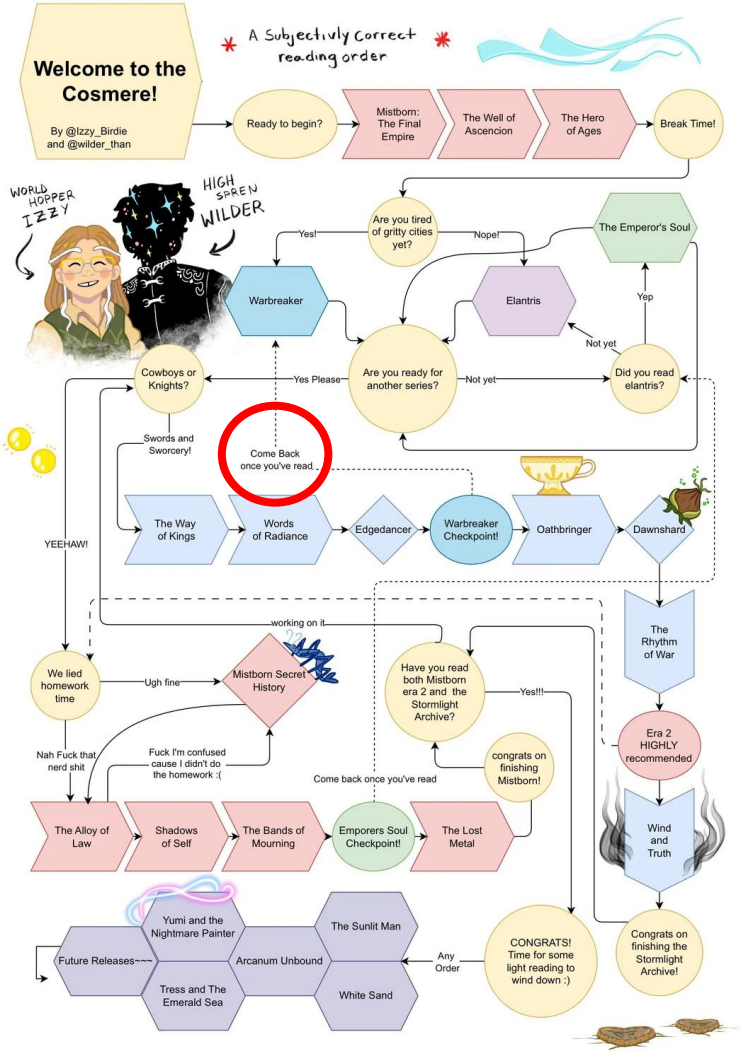
Brandon Sanderson



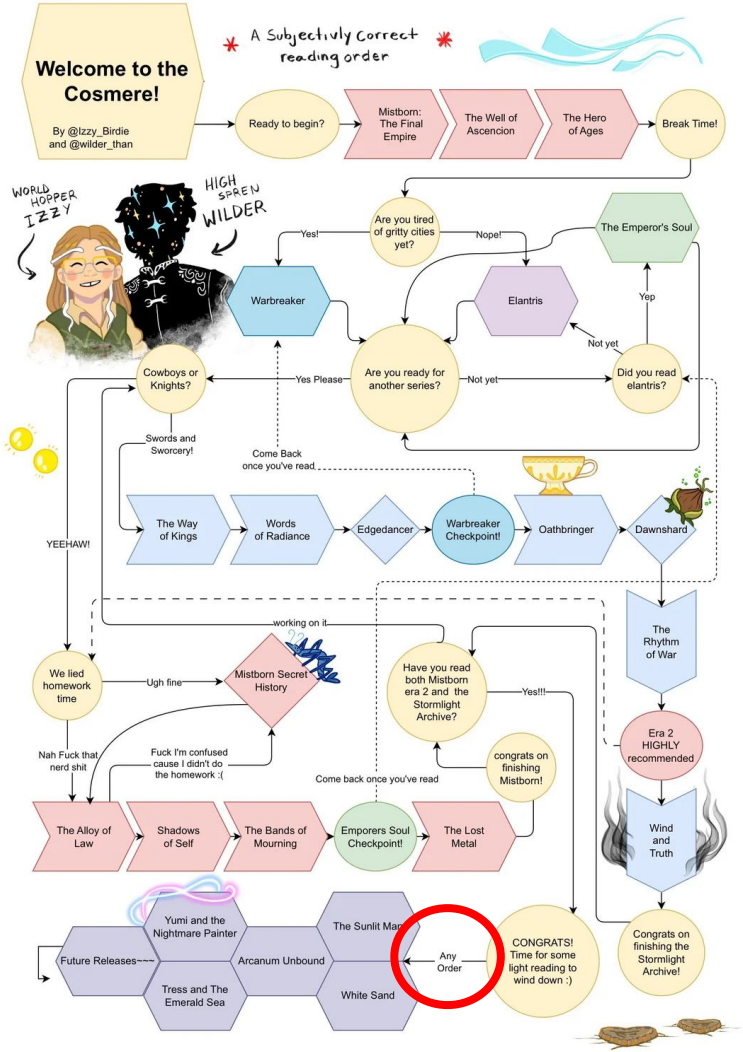
Brandon Sanderson



Brandon Sanderson

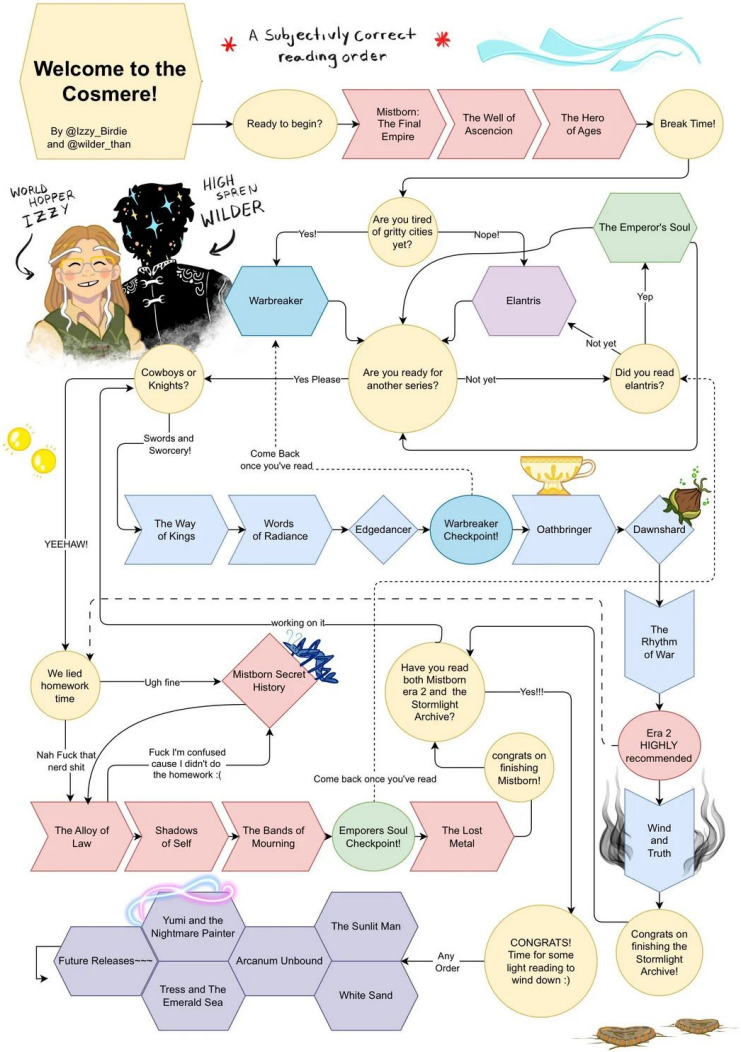


Brandon Sanderson



Brandon Sanderson

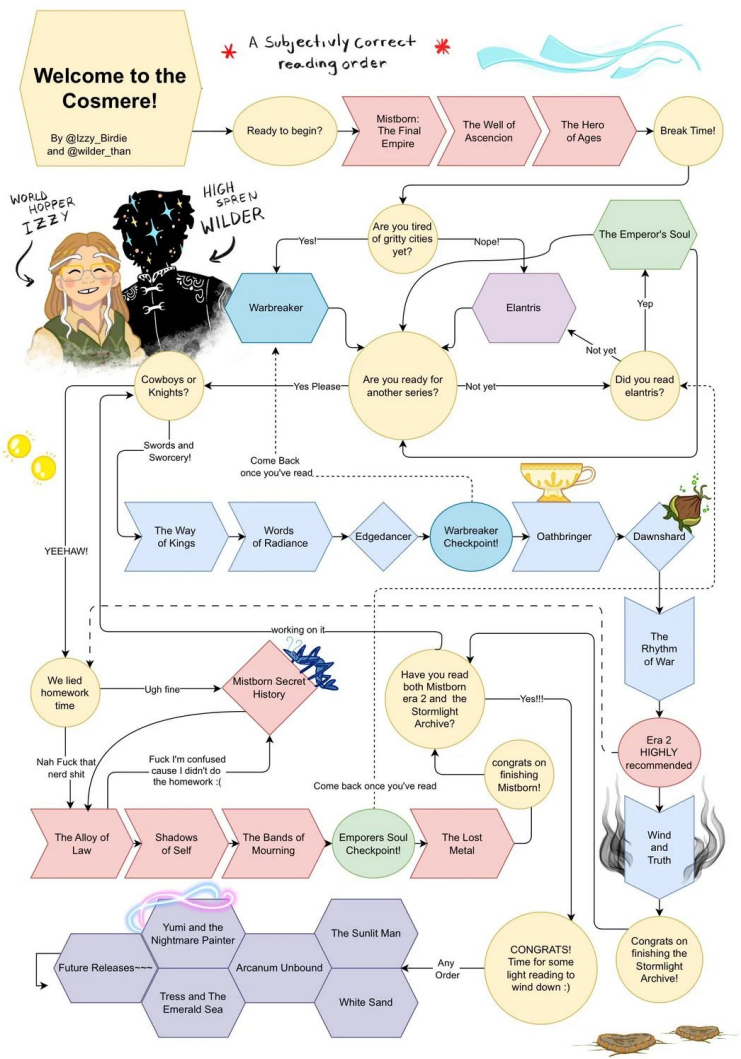
What does this remind you of?



Brandon Sanderson

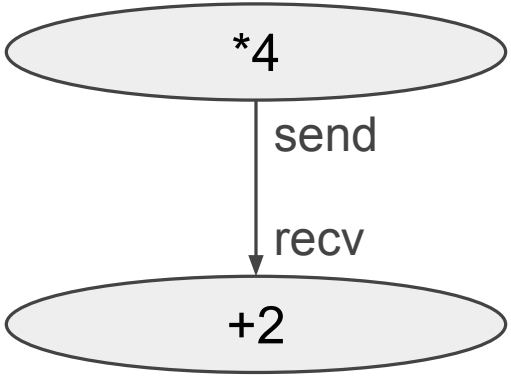
What does this remind you of?

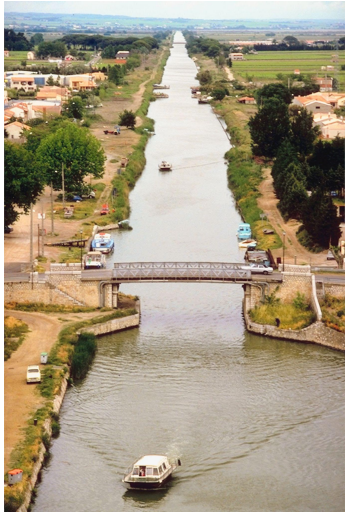
Dependency Graphs!



Channels

- How you implement dependencies
- Synchronize data transfer between coroutines!
  - send and recv





Example 3

```
fn main() {
    h, c = spawn(my_routine)
    send(c, 5)
    join(h)
}
```

```
fn my_routine(c) {
    x = recv(c)
    print(x)
}
```

Example 3

```
fn main() {
    PC
    h, c = spawn(my_routine)

    send(c, 5)

    join(h)

}
```

```
fn my_routine(c) {

    x = recv(c)

    print(x)

}
```

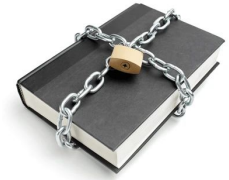
Example 3

```
fn main() {

    h, c = spawn(my_routine)

    send(c, 5)

    join(h)

}
```

```
fn my_routine(c) {
    PC
    x = recv(c)

    print(x)

}
```

Example 3

```
fn main() {

    h, c = spawn(my_routine)
    PC
    send(c, 5)

    join(h)

}
```



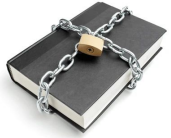
```
fn my_routine(c) {

    x = recv(c)

    print(x)

}
```

Example 3

```
fn main() {

    h, c = spawn(my_routine)

    send(c, 5)
    PC
    join(h)

}
```

```
fn my_routine(c) {

    x = recv(c)

    print(x)

}
```

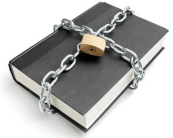
Example 3



```
fn main() {

    h, c = spawn(my_routine)

    send(c, 5)

    join(h)

}
```

```
fn my_routine(c) {

    x = recv(c)
    PC
    print(x)

}
```

Example 3



```
fn main() {

    h, c = spawn(my_routine)

    send(c, 5)

    join(h)

}
```

```
fn my_routine(c) {

    x = recv(c)

    print(x)
```

PC

```
}
```

Example 3

```
fn main() {

    h, c = spawn(my_routine)

    send(c, 5)

    join(h)
```

PC

```
}
```

```
fn my_routine(c) {

    x = recv(c)

    print(x)

}
```

Example 4

```
fn main() {
    h, c = spawn(numbers)
    for i in 0..5 {
        x = recv(c)
        print(x)
    }

}
```

```
fn numbers(c) {
    x = 0; y = 1;
    loop {
        sum = x + y
        send(c, sum)
        x = y
        y = sum
    }
}
```

Many flavors of channels

- Are send/recv buffered?
  - Can you send into a buffer (to be matched with a receiver later), or do you need a matching receiver immediately to finish the transaction?
    - HW1 is the latter (non-buffered channels)
- What type of data do they send?
  - In HW1, it will be a (void*), aka a pointer to anything
- What type of data is safe to send?

Communication without channels is unsafe

```
my_global = 1

fn main() {

    h, c = spawn(my_routine)

    print(my_global)

    join(h)

}
```

```
fn my_routine(c) {

    my_global = 2

}
```

Channels create causal consistency

If a program proceeds past a send/recv, then we know something about where 2 coroutines are

Channels create causal consistency

```
my_global = 1

fn main() {

    h, c = spawn(my_routine)

    _ = recv(c)

    print(my_global)

    join(h)

}
```

```
fn my_routine(c) {

    my_global = 2

    send(c, null)

    ...

}
```

Channels create causal consistency

Say we want to create 100 coroutines, but we want **only** main() to run until we are ready to “release” all of the coroutines. How do we do this?

- Hint: you can make arbitrary channels via create_channel()
- Hint: you can do this in O(1) space!

Channels create causal consistency

```
fn main() {
    big_c = create_channel()
    for i in 0..100 {
        h, c = spawn(my_routine)
        send(c, big_c)
    }
    ... once ready:
    for i in 0..100 {
        send(big_c, 1);
    }
}
```

```
fn my_routine(c) {
    big_c = recv(c)
    _ = recv(big_c)
    ... do work here
}
```

Why use Coroutines?

Coroutines sometimes could run in parallel (e.g. Go) (but not homework 1)

Coroutines allow you to program things in a more natural way

- Each function can take care of a specific task, but execution can return to main without the function needing to fully finish

Coroutines are cheap

- Can run hundreds of thousands of coroutines simultaneously

Coroutines enable asynchronous behavior

Why use Coroutines?

“Parallelize Waiting”

While waiting for a disk I/O or network API calls, you can do something else

Relationship to Async/Await

- Async/Await abstraction builds on top of coroutines
  - Automatically places yields, sometimes hides sending and receiving too

Go

One of the most popular emerging programming languages

Their gimmick is coroutines, which they playfully call “goroutines”

Go

`f(x)` to run f

`go f(x)` to run f in its own coroutine

Go

```go
func main() {
    ch := make(chan int)
    go worker(ch)

    r := <-ch
}
```

```go
func worker(ch chan int) {
    fmt.Print("working...")
    time.Sleep(time.Second)
    fmt.Println("done")

    ch <- 42
}
```

And now…

You’re basically a Go expert!*

But the principle of Coroutines applies in many languages

Now…

We will talk about the implementation of a Coroutine in C. It will be highly relevant to Homework 1.